



SAPIENZA
UNIVERSITÀ DI ROMA

Faculty of Information Engineering, Informatics and
Statistics
Department of Computer Science

Advanced Architectures

Author:
Simone Lidonnici

4 october 2026

Contents

E	Exercises	1
E.1	Exercises on performance	1
E.1.1	Amdahl's Law	1
E.1.2	Gustafson Law	2
E.1.3	Processor Performance	2
E.1.4	Circuit area and time	3
E.2	Exercises on number representation	5
E.2.1	Multiplication of two complement number	5
E.2.2	Sum with redundant number system	5
E.2.3	Simmetric number representation	6
E.2.4	Conversion between decimal and signed digit	7
E.2.5	Modified binary signed digit	7
E.2.6	Redundant binary	8
E.2.7	Residue number system	9
E.2.8	Multiple radix system	10
E.3	Exercises on sparse matrices	11
E.3.1	COO format	12
E.3.2	Skyline format	12
E.3.3	Compressed Sparce Row (CSR)/Column (CSC)	13
E.3.4	Modified Sparse Row (MSR)	14
E.3.5	Block Sparse Row (BSR)	14
E.3.6	Diagonal format	15
E.3.7	Ellpack-Itpack	16
E.4	Exercises on networks	17
E.4.1	Crossbar and $\log n$ MIN	17
E.4.2	CLOS network	18
E.4.3	Benes network	20
E.4.4	Extended generalized fat tree (XGFT)	21
E.5	Exercises on quantum computing	22
E.5.1	System of two qubit	22
E.5.2	Quantum circuits	23

E

Exercises

E.1 Exercises on performance

E.1.1 Amdahl's Law

Amdahl's Law

The Amdahl's law gives a bound on the speedup of a program that is enhanced (parallelized) based on the fraction of program that we can enhance α . We can calculate the time with the enhancement:

$$T_e = (1 - \alpha)T + \alpha \frac{T}{S_e}$$

In which T_e is the time with the enhancement, S_e the speedup of the enhanced part and T the base program time. For parallelization with p process we can assume $S_e = p$. We can calculate the speedup as:

$$S = \frac{1}{(1 - \alpha) + \frac{\alpha}{S_e}}$$

Another important metric is the efficiency with p processes:

$$E = \frac{S}{p} = \frac{1 - \alpha}{p} + \alpha$$

Example:

We have a processor that does:

- 20% of the time FPSQR
- 50% of time FP

We have 2 option of enhancement:

1. Speedup FPSQR by 10 times
2. Speedup FP by 1.6 times

We calculate the speedup of the two options:

$$S_1 = \frac{1}{(1 - 0.2) + \frac{0.2}{10}} = \frac{1}{0.82} \simeq 1.22$$

$$S_2 = \frac{1}{(1 - 0.5) + \frac{0.5}{1.6}} = \frac{1}{0.8125} \simeq 1.23$$

So, the second one is slightly better.

E.1.2 Gustafson Law

Gustafson Law

The Gustafson Law uses the opposite approach in respect to the Amdahl law, so using enhanced time (parallelized) T_e with p processes, we can estimate the sequential time based on the fraction of program that we can enhance α :

$$T = (1 - \alpha)T_e + \alpha p T_e$$

The speedup can be calculated:

$$S = (1 - \alpha) + \alpha p$$

Example:

We have a program that can be parallelized for 50% and we have two processors, one with 2 core and the other with 4. The speedup that the two processors can reach is:

$$S_1 = (1 - 0.5) + 2 \cdot 0.5 = 1.5 \implies E_1 = \frac{1.5}{2} = 0.75$$

$$S_2 = (1 - 0.5) + 4 \cdot 0.5 = 2.5 \implies E_2 = \frac{2.5}{4} = 0.625$$

So, with 4 cores we have more speedup but less efficiency.

E.1.3 Processor Performance

Processors uses clock cycles (CC) to run at constant rate. For a program we can calculate the CPU time for a program as:

$$CPU \text{ time} = CC \text{ program} / C \text{ rate} = CC \text{ program} \cdot C \text{ time}$$

We can count the number of instruction executed (IC) and knowing the CC per program we can calculate the CC per instruction (CPI):

$$CPI = C \text{ program} / IC$$

With the previous formulas we can redefine the CPU time as:

$$CPU \text{ time} = IC \cdot CPI \cdot C \text{ time}$$

It is useful to also calculate the total clock cycle of a program considering the different types of instructions:

$$CC \text{ program} = \sum_{\forall i} IC_i \cdot CPI_i$$

In which i is a type of instruction.

If on a processor we decrease the CPI of a specific instruction, we can calculate the new CPI by subtracting the weighted difference:

$$CPI_e = CPI - \%instr(CPI_{old-instr} - CPI_{new-instr})$$

In which $\%instr$ is the fraction of the instruction enhanced, $CPI_{old-instr}$ is the CPI of that specific instruction before the enhancement and $CPI_{new-instr}$ after the enhancement.

Example:

We have a processor that does:

- 2% of the time FPSQR with 20 CPI
- 25% of time FP with 4 CPI
- Other instruction with 1.33 CPI

We have 2 option of enhancement:

1. Decrease the CPI of FPSQR to 2
2. Decrease the CPI of FP to 2.5

The starting CPI average is:

$$CPI_{old} = (4 \cdot 0.25) + (1.33 \cdot 0.75) = 2$$

The enhanced CPI and speedup are:

$$CPI_1 = 2 - 0.02(20 - 2) \simeq 1.64 \implies S_1 = \frac{2}{1.64} \simeq 1.22$$

$$CPI_2 = (2.5 \cdot 0.25) + (1.33 \cdot 0.75) \simeq 1.625 \implies S_2 = \frac{2}{1.625} \simeq 1.23$$

So, the second one is slightly better.

E.1.4 Circuit area and time

To approximate the time and area of a circuit we relwte them to the area and speed of the simplest gates. Each simple gate with 2 input count as 1 gate for area and 1 gate delay unit for time. The only exception is the xor gate which counts as 2 because is implemented with 2 basic gates.

To take in account the number of inputs of gates we count a m input gate as:

- Time: $\lceil \log m \rceil$ gates
- Area: $m - 1$ gates

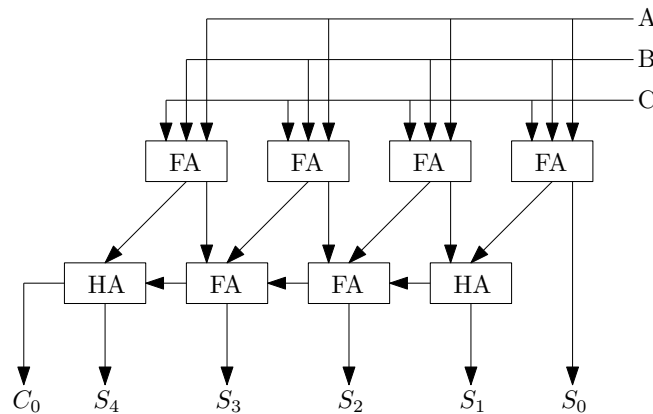
From now on, we will define the gate unit for area as A and the gate delay unit for time as T. Some simple adders have this costs:

1. Half adder:

- Time: $2T$
 - Area: $3A$
2. Full adder:
- Time: $4T$
 - Area: $7A$
3. Ripple-Carry adder with n bits:
- Time: $4nT$
 - Area: $7nT$

Example:

We have 3 operands of 4 bits and we want to sum them using the Carry-Save Adder below:



We can calculate the time and area of this adder in respect to the RCA for the same 3 4-bits operands:

1. Carry-Save Adder:
- Time: $FA + HA + 2FA + HA = 4 + 2 + 2 \cdot 4 + 2 = 16T$
 - Area: $6FA + 2HA = 6 \cdot 7 + 2 \cdot 3 = 48A$
2. Ripple-Carry:
- Time: $2RCA (4 \text{ bits}) = 2 \cdot 4 \cdot 4 = 32T$
 - Area: $RCA (4\text{bits}) = 7 \cdot 4 = 28A$ (we can reuse the same two times)

The speedup for both time and area is:

- Time: $\frac{32}{16} = 2$
- Area: $\frac{28}{48} \simeq 0.6$

E.2 Exercises on number representation

E.2.1 Multiplication of two complement number

To multiply two operands written in two's complement we can use a easier trick in respect to doing the multiplication the normal way. Let A and B be the two operands we want to multiply, for every digit a_i in A :

- If (a_i, a_{i-1}) is $(0,0)$ or $(1,1)$: nothing to add
- If (a_i, a_{i-1}) is $(1,0)$ or the firsts digit as 1: sub B
- If (a_i, a_{i-1}) is $(0,1)$: add B

The add and sub are shifted based on the position of the i -th digit in A . Also each sub or add must be written as if the number has $|A| + |B|$ bits.

Example:

We want to multiply 10 and -7, so in two's complement:

$$A = 01010 \quad B = 11001$$

The multiplication is done:

							0	1	0	1	0
x							1	1	0	0	1
-	0	0	0	0	0	0	0	1	0	1	0
+	0	0	0	0	0	0	1	0	1	0	/
+											/
-	0	0	0	1	0	1	0	/			

The sub are transformed into sum of the opposite, so the complement plus 1. The number of digits of the result is the sum of digits of the two numbers.

							0	1	0	1	0
x							1	1	0	0	1
+	1	1	1	1	1	1	0	1	1	0	
+	0	0	0	0	0	0	1	0	1	0	/
+											/
+	1	1	1	0	1	1	0	/			
1	1	1	1	0	1	1	1	0	1	0	

The result is correct, as the result is -70 .

E.2.2 Sum with redundant number system

We can have redundant number system where the number of digit allowed for a particular radix is higher than the radix itself.

To sum two number we must:

1. Sum the numbers digit by digit, creating a result that has a digit set doubled than the initial two numbers.
2. Define an intermediate sum and carry that sum up to the initial digit set, generating the carry using the radix.
3. Sum the intermediate sum with the carry.

Example:

We have a number system in radix 10 and digit set $[0, 11]$. The two number we want to sum are:

$$\begin{array}{rcccccccc}
 & 11 & 10 & 7 & 11 & 3 & 8 & \text{digit} \in [0, 11] \\
 + & 7 & 2 & 9 & 10 & 9 & 8 & \text{digit} \in [0, 11] \\
 \hline
 & 18 & 12 & 16 & 21 & 12 & 16 & \text{digit} \in [0, 22] \\
 & 8 & 2 & 6 & 1 & 2 & 6 & \text{digit} \in [0, 9] \\
 & 1 & 1 & 1 & 2 & 1 & 1 & / \quad \text{digit} \in [0, 2] \\
 \hline
 & 1 & 9 & 3 & 8 & 2 & 3 & 6 \quad \text{digit} \in [0, 11]
 \end{array}$$

E.2.3 Simmetric number representation

Usually redundant number systems with radix r have a digit set in the form $[\bar{a}, a]$ where $\left\lceil \frac{r-1}{2} \right\rceil \leq a \leq r-1$.

We can calculate the redundancy of the digit set by dividing the number of numbers that have more than one representation by the total number in the range of digits for a defined number of digits n . For a range of digits $[-\alpha, \beta]$ we can calculate the redundancy index ρ as:

$$\rho = \alpha + \beta + 1 - r$$

This digit sets permit to do the sum using an algorithm:

$$(x_{n-1}, \dots, x_0) \pm (y_{n-1}, \dots, y_0) = (s_{n-1}, \dots, s_0)$$

1. Compute c_i with the rule:

$$c_i = \begin{cases} 1 & (x_i + y_i) \geq a \\ \bar{1} & (x_i + y_i) \leq \bar{a} \\ 0 & \bar{a} < (x_i + y_i) < a \end{cases}$$

2. Compute the interim sum u_i :

$$u_i = x_i + y_i - rc_i$$

3. Compute s_i :

$$s_i = u_i + c_{i-1}$$

Example:

We have a number system with $r = 10$ and $a = 6$.

With $n = 2$ the numbers in the range $\bar{66} \leq x \leq 66$ are 133, but we have $13^2 = 169$ number representation, so the redundancy is:

$$(169 - 133)/133 = 27\%$$

The redundant index is:

$$\rho = 6 + 6 + 1 - 10 = 3$$

We want to sum two numbers $X = 3645$ and $Y = 1456$:

		3	6	4	5	X
+		1	4	5	6	Y
	0	1	1	1		C
		4	0	$\bar{1}$	1	U
		5	1	0	1	S

E.2.4 Conversion between decimal and signed digit

Using the digit set as before $[\bar{a}, a]$ with radix $r = 10$, we can easily convert a number from decimal to this representation.

- Decimal to signed digit: Using the same formula as the sum but backwards, calculating the carry and interim sum.
- Signed digit to decimal: Simply subtract the negative digits.

Example:

We have the number 6849 in decimal and the conversion in signed digit with digit set $[\bar{6}, 6]$ is:

		6	8	4	9	X+Y
1	1	0	1			C
		$\bar{4}$	$\bar{2}$	4	$\bar{1}$	U
1	$\bar{3}$	2	5	$\bar{1}$		S

The conversion backwards is simply:

	1	$\bar{3}$	$\bar{2}$	5	$\bar{1}$
	1	0	0	5	0
-	0	3	2	0	1
		6	8	4	9

E.2.5 Modified binary signed digit

In binary we can only have $a = 1$, so the digit set can only be $\{\bar{1}, 0, 1\}$, to perform the sum of two numbers in this representation 3 steps are needed:

1. The two lines are the sum and carry but if the result is 1 its written as $1\bar{1}$ and if the result is $\bar{1}$ is written as $\bar{1}1$. This can be done because of the redundant system.
2. The two lines are simply sum and carry
3. Same as the first

At the end you will have the sum and complemented sum in the last two lines.

Example:

If the two numbers we want to sum are 9, written as $1\bar{1}01\bar{1}$ and -10 , written as $\bar{1}1\bar{1}10$, the sum is done:

$$\begin{array}{rcccccc}
 & 1 & \bar{1} & 0 & 1 & \bar{1} & 9 \\
 & \bar{1} & 1 & \bar{1} & 1 & 0 & -10 \\
 \hline
 & 0 & 0 & 1 & 0 & 1 & \\
 0 & 0 & \bar{1} & 1 & \bar{1} & / & \\
 \hline
 & 0 & \bar{1} & 0 & \bar{1} & 1 & \\
 0 & 0 & 1 & 0 & 0 & / & \\
 \hline
 & 0 & 0 & 0 & 1 & \bar{1} & 1 \\
 0 & 0 & 0 & \bar{1} & 1 & & -1
 \end{array}$$

E.2.6 Redundant binary

Exists another binary redundant system in which each power of 2 is defined by 2 digit that are summed, for example 01 11 is equal to $0 \cdot 2^1 + 1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^0 = 4$.

To make the sum the follow table is used, doing 2 iterations (the upper operand is the column and the lower one the row):

	00	01	10	11
00	00 00	10 00	00 01	10 01
01	00 01	10 01	00 10	10 10
10	00 01	10 01	00 10	10 10
11	00 10	10 10	00 11	10 11

A trick to remember the table is to write the upper result of the 1st and 3rd column as 00 (the ones with 00 and 10 on top) and the upper result of the 2nd and 4th column as 10 (the ones with 01 and 11 on top). After that the lower result can be computed simply by doing the sum. The representation of negative number follows the same idea of the two's complement in which the first digit (in this case the first pair) is negative. To perform the complement we reverse each bit and we add 2 (11 as the first digit).

Example:

We want to perform $A - B$ in which $A = 00\ 01\ 00\ 01\ 10$ and $B = 00\ 00\ 01\ 11\ 11$.

We convert B into its complement to perform the sub:

$$\begin{array}{rcccccc}
 11 & 11 & 10 & 00 & 00 & + \\
 & & & 01 & 00 & = \\
 \hline
 11 & 11 & 10 & 01 & 00 &
 \end{array}$$

The sum between $A - B$ is done as follows:

$$\begin{array}{rcccccc}
& 00 & 01 & 00 & 01 & 10 & + \\
& 11 & 11 & 10 & 00 & 11 & \\
\hline
& 00 & 10 & 00 & 10 & 00 & \\
1 & 01 & 00 & 10 & 01 & 1/ & \\
\hline
& 00 & 00 & 00 & 00 & 00 & \\
0 & 10 & 10 & 11 & 00 & 1/ &
\end{array}$$

This result is correct because the numbers are:

1. $A = 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 2 + 1 = 11$
2. $B = 1 \cdot 2^2 + 2 \cdot 2^1 + 2 \cdot 2^0 = 4 + 4 + 2 = 10$
3. $A - B = -1 \cdot 2^4 + 1 \cdot 2^3 + 2 \cdot 2^2 + 1 \cdot 2^0 = -16 + 8 + 8 + 1 = 1$

E.2.7 Residue number system

A residue number system is a representation based on a set of moduli $S = \{m_1, \dots, m_n\}$ that define a range $[0, M-1]$ where $M = m_1 \cdot \dots \cdot m_n$. A number X in this number system is written as:

$$X = \langle x_1, \dots, x_n \rangle$$

where $x_i = X \bmod m_i$.

If the number that compose the module set are relatively prime, every number $X < M$ has a unique representation.

To represent also negative number we change the range of numbers:

1. If M is odd: $\left[-\frac{(M-1)}{2}, \frac{M-1}{2}\right]$
2. If M is even: $\left[-\left(\frac{(M-1)}{2} - 1\right), \frac{M}{2}\right]$

With negative number the value of a number in respect to a specific module m_i is:

$$x_i = \begin{cases} \langle X \rangle_{m_i} & X \geq 0 \\ \langle m_i - \langle |X| \rangle_{m_i} \rangle_{m_i} & X < 0 \end{cases}$$

In this number system the basic operation are done as follows:

- Sum: add each module component of the two number and apply the module at the end.

$$X + Y = Z \quad Z_i = \langle X_i + Y_i \rangle_{m_i}$$

- Subtraction: add the first number and the opposite of the second one. The opposite of a number is the additive inverse of each module component:

$$X_i + \overline{X}_i = m_i$$

- Multiplication: multiply each module component of the two number and apply the module at the end.

$$X \cdot Y = Z \quad Z_i = \langle X_i \cdot Y_i \rangle_{m_i}$$

- Division: multiply the first number and the multiplicative inverse of the second one. The multiplicative inverse is the number that multiplied by the original number gives 1:

$$\langle X_i \cdot X_i^{-1} \rangle_{m_i} = 1$$

Example:

If we have a module set $S = \{2, 3, 5\}$ with value of $M = 2 \cdot 3 \cdot 5 = 30$, the range represented will be $[-14, 15]$.

Two number 14 and 7 converted in this number system will be written as:

$$14 = \langle 0, 2, 4 \rangle \quad 7 = \langle 1, 1, 2 \rangle$$

The different operation between the two numbers can be done:

1. Sum: $14 + 7 = \langle 0, 2, 4 \rangle + \langle 1, 1, 2 \rangle = \langle 1, 3, 6 \rangle = \langle 1, 0, 1 \rangle$
2. Multiplication: $14 \cdot 7 = \langle 0, 2, 4 \rangle \cdot \langle 1, 1, 2 \rangle = \langle 0, 2, 8 \rangle = \langle 0, 2, 3 \rangle$
3. Subtraction: first we need to calculate the inverse of 7.

$$7 - 7 = 0 \implies -7 = \langle 1, 2, 3 \rangle$$

And then the subtraction is simply:

$$14 - 7 = \langle 0, 2, 4 \rangle + \langle 1, 2, 3 \rangle = \langle 1, 4, 7 \rangle = \langle 1, 1, 2 \rangle$$

4. Division: first we need to calculate the multiplicative inverse of 5.

$$7 \cdot 7^{-1} = 1 \implies 7^{-1} = \langle 1, 1, 3 \rangle$$

And then the division is simply:

$$14/7 = \langle 0, 2, 4 \rangle \cdot \langle 1, 1, 3 \rangle = \langle 0, 2, 12 \rangle = \langle 0, 2, 2 \rangle$$

E.2.8 Multiple radix system

Given a module set $S = \{m_n, \dots, m_1\}$ a number in the multiple radix system is represented as:

$$X = x_n \cdot (m_{n-1} \cdot \dots \cdot m_1) + \dots + x_3 \cdot (m_2 \cdot m_1) + x_2 \cdot m_1 + x_1$$

To convert a number written in RNS $X = \langle x_n, \dots, x_1 \rangle$ to MRS we need to follow a series of steps (we call x the RNS representation and z the MRS representation):

1. z_1 is equal to x_1
2. For each other digit z_i :
 - 2.1 Subtract $X - z_{i-1}$
 - 2.2 Divide X by m_{i-1}
 - 2.3 The i -th digit of the result will be z_i

Example:

Given the module set $S = \{13, 11, 8, 7\}$ and the number $X = 68$ we want to write the RNS representation and convert it in the MRS.

The RNS representation is:

$$X = \langle 3, 2, 4, 5 \rangle$$

The conversion follows this steps:

$$1. \ z_1 = x_1 = 5$$

$$2. \ z_2:$$

$$2.1 \ X - z_1 = X - 5 = \langle 11, 8, 7, - \rangle$$

$$2.2 \ 7^{-1} = \langle 2, 8, 7, - \rangle \implies X/7 = \langle 9, 9, 1, - \rangle$$

$$2.3 \ z_2 = 1$$

$$3. \ z_3:$$

$$3.1 \ X - z_2 = X - 1 = \langle 8, 8, -, - \rangle$$

$$3.2 \ 8^{-1} = \langle 5, 7, -, - \rangle \implies X/8 = \langle 1, 1, -, - \rangle$$

$$3.3 \ z_3 = 1$$

$$4. \ z_4:$$

$$4.1 \ X - z_3 = X - 1 = \langle 0, -, -, - \rangle$$

$$4.2 \ 11^{-1} = \langle 6, -, -, - \rangle \implies X/11 = \langle 0, -, -, - \rangle$$

$$4.3 \ z_4 = 0$$

So, X written as MRS is:

$$Z = (0, 1, 1, 5)$$

We can control if its right by doing the basic calculation:

$$Z = 0 \cdot (11 \cdot 8 \cdot 7) + 1 \cdot \underbrace{(8 \cdot 7)}_{56} + 1 \cdot 7 + 5 = 68$$

E.3 Exercises on sparse matrices

All the exercises on sparse matrix will consider the following:

- n is the matrix size ($n \times n$ matrix)
- nz is the number of non-zero values
- The indexes start from 1

For the representation we consider:

- Real number: 8 byte
- Integer: 4 byte

So a standard matrix will use $8 \cdot n^2$ B.

E.3.1 COO format

The COO format uses 3 array:

- A : array to contain all non-zero value
- I : array containing the row indexes
- J : array containing the column indexes

This representation occupies:

$$(8 + 4 + 4) \cdot nz \text{ B}$$

Example:

Given the matrix:

$$\begin{vmatrix} 1 & 0 & 0 & 2 & 0 \\ 3 & 4 & 0 & 5 & 0 \\ 6 & 0 & 7 & 8 & 9 \\ 0 & 0 & 10 & 11 & 0 \\ 0 & 0 & 0 & 0 & 12 \end{vmatrix}$$

The standard representation occupies:

$$8 \cdot 5^2 = 200 \text{ B}$$

The COO format representation occupies:

$$(8 + 4 + 4) \cdot 5 = 80 \text{ B}$$

E.3.2 Skyline format

The Skyline format stores all values between the first and the last non-zero values on each row. There are difference based on if the matrix is symmetric or not:

1. Symmetric matrix:

- D : array to contain all diagonal values
- AL : array containing all values between first and last non-zero values on each row of the lower triangular part of the matrix
- I : array containing in the k -th element the index of the first element of row $k + 1$ in AL

This representation occupies:

$$(n_{AL} + n) \cdot 8 + n \cdot 4 \text{ B}$$

2. General matrix:

- D : array to contain all diagonal values
- E : array containing the lower triangular part
- F : array containing the upper triangular part column-wise

- I : array containing in the k -th element the index of the first element of row $k + 1$ in E
- J : array containing in the k -th element the index of the first element of column $k + 1$ in F (if the pattern of the matrix is symmetric this array is not needed)

This representation occupies:

$$(n_E + n_F + n) \cdot 8 + \underbrace{2}_{\text{if } J \text{ exists}} \cdot n \cdot 4 \text{ B}$$

Example:

Given the matrix:

$$\begin{vmatrix} 1 & 4 & 0 & 13 & 0 & 0 & 0 \\ 4 & 5 & 9 & 14 & 18 & 0 & 0 \\ 0 & 9 & 10 & 0 & 19 & 25 & 0 \\ 13 & 14 & 0 & 15 & 20 & 0 & 30 \\ 0 & 18 & 19 & 20 & 21 & 26 & 31 \\ 0 & 0 & 25 & 0 & 26 & 27 & 0 \\ 0 & 0 & 0 & 30 & 31 & 0 & 32 \end{vmatrix}$$

The standard representation occupies:

$$8 \cdot 7^2 = 392 \text{ B}$$

The skyline format representation for symmetric matrices occupies:

$$(14 + 7) \cdot 8 + 7 \cdot 4 = 196 \text{ B}$$

The skyline format considering it a regular matrix:

$$\begin{cases} (14 + 14 + 7) \cdot 8 + 7 \cdot 4 = 308 \text{ B} & \text{symmetric pattern} \\ (14 + 14 + 7) \cdot 8 + 2 \cdot 7 \cdot 4 = 336 \text{ B} & \text{otherwise} \end{cases}$$

E.3.3 Compressed Sparse Row (CSR)/Column (CSC)

The CSR format uses 3 array:

- A : array to contain all non-zero value
- I : array containing in the k -th element the index of the first element of row k in A and J
- J : array containing the column indices for each value in A

This representation occupies:

$$8 \cdot nz + 4 \cdot (nz + n + 1) \text{ B}$$

The CSC exchange the role of the arrays I and J and stores the values column-wise, having the same space occupied.

Example:

Given the matrix:

$$\begin{vmatrix} 1 & 4 & 0 & 13 & 0 & 0 & 0 \\ 4 & 5 & 9 & 14 & 18 & 0 & 0 \\ 0 & 9 & 10 & 0 & 19 & 25 & 0 \\ 13 & 14 & 0 & 15 & 20 & 0 & 30 \\ 0 & 18 & 19 & 20 & 21 & 26 & 31 \\ 0 & 0 & 25 & 0 & 26 & 27 & 0 \\ 0 & 0 & 0 & 30 & 31 & 0 & 32 \end{vmatrix}$$

The standard representation occupies:

$$8 \cdot 7^2 = 392 \text{ B}$$

The CSR or CSC formats occupy:

$$8 \cdot 29 + 4 \cdot (29 + 7 + 1) = 340 \text{ B}$$

E.3.4 Modified Sparse Row (MSR)

The MSR format uses 2 array:

- V : array containing in the first n elements the diagonal, the $n + 1$ element is empty and from the $n + 2$ element all the non-zero values except the diagonal are stored row-wise
- B : array containing in the first $n + 1$ values the indexes where the rows begin in V and from $n + 2$ each k -th value represent the column index of the k -th value in V

This representation occupies:

$$(8 + 4) \cdot (nz + 1) \text{ B}$$

Example:

Given the matrix:

$$\begin{vmatrix} 1 & 4 & 0 & 13 & 0 & 0 & 0 \\ 4 & 5 & 9 & 14 & 18 & 0 & 0 \\ 0 & 9 & 10 & 0 & 19 & 25 & 0 \\ 13 & 14 & 0 & 15 & 20 & 0 & 30 \\ 0 & 18 & 19 & 20 & 21 & 26 & 31 \\ 0 & 0 & 25 & 0 & 26 & 27 & 0 \\ 0 & 0 & 0 & 30 & 31 & 0 & 32 \end{vmatrix}$$

The standard representation occupies:

$$8 \cdot 7^2 = 392 \text{ B}$$

The MSR format occupies:

$$(8 + 4) \cdot (29 + 1) = 360 \text{ B}$$

E.3.5 Block Sparse Row (BSR)

The BSR divides the matrix in squared blocks that evenly divide the matrix. The format stores all values of a non-zero block where a non-zero block is a block with at least one non-zero value.

The BSR format uses 4 array:

- *Values*: array containing the values of all non-zero blocks stored block-by-block in row order. Within each block the values are stored column-wise in case of one-based indexing or row-wise in case of zero-based indexing
- *Columns*: array where element i is the number of the column in the block matrix that contain the i -th non-zero block
- *PointerB*: array where element j is the index of the element in *Columns* that is the first non-zero block in row j of the block matrix
- *PointerE*: array where element j is the index of the element in *Columns* that is the last non-zero block in row j of the block matrix plus 1

This representation occupies:

$$n_{Values} \cdot 8 + 4 \cdot (n_{blocks} + 2 \cdot n_{row}) \text{ B}$$

Example:

Given the matrix:

$$\begin{vmatrix} 1 & 4 & 0 & 13 & 0 & 0 \\ 4 & 5 & 9 & 14 & 0 & 0 \\ 0 & 9 & 10 & 0 & 19 & 25 \\ 13 & 14 & 0 & 15 & 20 & 0 \\ 0 & 0 & 19 & 20 & 0 & 0 \\ 0 & 0 & 25 & 0 & 0 & 0 \end{vmatrix}$$

The standard representation occupies:

$$8 \cdot 6^2 = 288 \text{ B}$$

If we divide the matrix in 2×2 blocks we obtain:

1	4	0	13	0	0
4	5	9	14	0	0
0	9	10	0	19	25
13	14	0	15	20	0
0	0	19	20	0	0
0	0	25	0	0	0

The BSR representation occupies:

$$24 \cdot 8 + 4 \cdot (6 + 2 \cdot 6) = 264 \text{ B}$$

E.3.6 Diagonal format

The diagonal format is used when we have diagonally structured matrices, that are matrices where the non-zero element are all located along a small number of diagonals.

The Diagonal format uses 2 array:

- *DIAG*: rectangular array storing the Nd diagonals

- *IOFF*: array containing the offset of each diagonal in respect to the main diagonal

This representation occupies:

$$(8 \cdot n + 4) \cdot Nd \text{ B}$$

Example:

Given the matrix:

$$\begin{vmatrix} 1 & 4 & 0 & 0 & 0 & 0 \\ 4 & 5 & 9 & 0 & 0 & 0 \\ 0 & 9 & 10 & 0 & 0 & 0 \\ 0 & 0 & 7 & 15 & 12 & 0 \\ 0 & 0 & 0 & 20 & 0 & 0 \\ 0 & 0 & 0 & 0 & 21 & 8 \end{vmatrix}$$

The standard representation occupies:

$$8 \cdot 6^2 = 288 \text{ B}$$

There are 3 diagonals, so the Diagonal format occupies:

$$(8 \cdot 6 + 4) \cdot 3 = 156 \text{ B}$$

E.3.7 Ellpack-Itpack

The Ellpack-Itpack format uses 2 array:

- *COEF*: rectangular array containing in each row all the non-zero values of that row of the matrix. The width of this array is defined by the maximum number of non-zero values in a row Nd
- *JCOEF*: rectangular array containing the column index of each value in *COEF*

This representation occupies:

$$(8 + 4) \cdot n \cdot Nd \text{ B}$$

Example:

Given the matrix:

$$\begin{vmatrix} 1 & 4 & 0 & 0 & 0 & 0 \\ 4 & 5 & 9 & 0 & 0 & 0 \\ 0 & 9 & 10 & 0 & 0 & 0 \\ 0 & 0 & 7 & 15 & 12 & 0 \\ 0 & 0 & 0 & 20 & 0 & 0 \\ 0 & 0 & 0 & 0 & 21 & 8 \end{vmatrix}$$

The standard representation occupies:

$$8 \cdot 6^2 = 288 \text{ B}$$

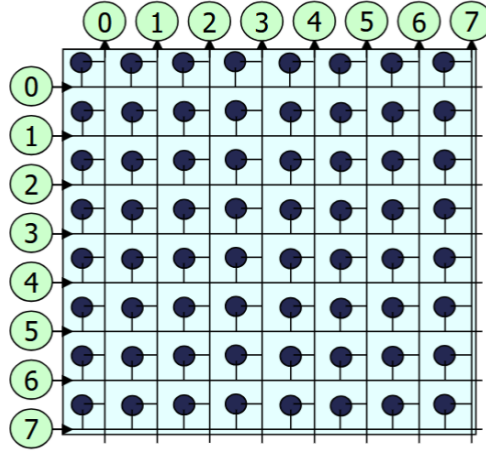
There are maximum 3 non-zero values in a row, so the Ellpack-Itpack format occupies:

$$(8 + 4) \cdot 6 \cdot 3 = 216 \text{ B}$$

E.4 Exercises on networks

E.4.1 Crossbar and $\log n$ MIN

The crossbar network is simply a grid in which all output are connected to all inputs.



The cost of the network in terms of switch and links is:

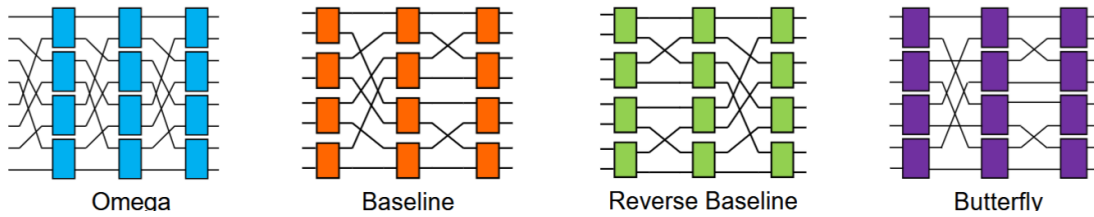
- Switches:

$$\text{cost}_{\text{switch}} = N^2$$

- Links:

$$\text{cost}_{\text{links}} = 2N$$

A $\log n$ MIN is a network with $\log n$ layer of switches, composed in different ways, such as:



The precise way in which the nodes are connected depends on how the switches are composed ($2 \times 2, 4 \times 4, \dots$). The cost of this type of networks given generic $k \times k$ switches in terms of switches and links is:

- Switches:

$$\text{cost}_{\text{switch}} = \frac{N}{k} \log_k N$$

- Links:

$$\text{cost}_{\text{links}} = N \cdot (\log_k N + 1)$$

Example:

Compute the cost of a crossbar and MIN with 4096 nodes and calculate the relative cost of MIN in respect to the crossbar network, considering switches with $k = 2, 4, 16$. Consider the cost of a switch quadratic in respect to the value k .

We start with the crossbar cost:

- Switches:

$$\text{cost}_{\text{switch}} = 4096^2$$

- Links:

$$\text{cost}_{\text{links}} = 2 \cdot 4096$$

Then we calculate the different relative cost depending on the k value of switches:

- $k = 2$:

- Switches:

$$\text{cost}_{\text{switch}} = \frac{4096^2}{2^2 \cdot \frac{4096}{2} \log_2 4096} = 170$$

- Links:

$$\text{cost}_{\text{links}} = \frac{2 \cdot 4096}{4096 \cdot (\log_2 4096 + 1)} = 0.15$$

- $k = 4$:

- Switches:

$$\text{cost}_{\text{switch}} = \frac{4096^2}{4^2 \cdot \frac{4096}{4} \log_4 4096} = 170$$

- Links:

$$\text{cost}_{\text{links}} = \frac{2 \cdot 4096}{4096 \cdot (\log_4 4096 + 1)} = 0.28$$

- $k = 16$:

- Switches:

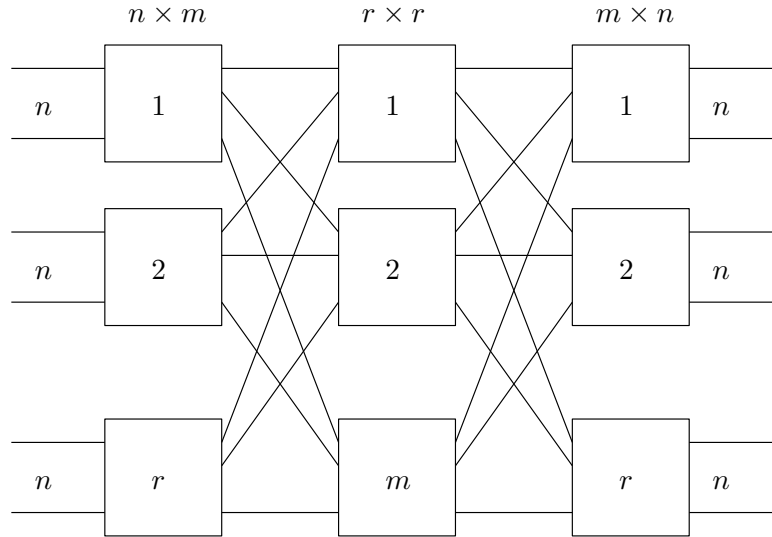
$$\text{cost}_{\text{switch}} = \frac{4096^2}{16^2 \cdot \frac{4096}{16} \log_{16} 4096} = 85$$

- Links:

$$\text{cost}_{\text{links}} = \frac{2 \cdot 4096}{4096 \cdot (\log_{16} 4096 + 1)} = 0.5$$

E.4.2 CLOS network

A CLOS network is made by three column of switches of different sizes, following the structure:



This network can be strictly or rearrangeable non-blocking based on the value of m :

1. Strictly non-blocking: $m \geq 2n - 1$
2. Rearrangeable non-blocking: $m \geq n$

The cost of the network in terms of switches and links is:

- Switches:

$$\text{cost}_{\text{switch}} = 2r + m$$

If some of the switches are composed by putting together minor switches, the value of m or r must be multiplied by the number of sub-switches that compose that specific type of switch.

- Links:

$$\text{cost}_{\text{links}} = 2mr$$

Example:

Design a Clos network of size 475×475 , using only square modules of size 16. Consider both the cases of strictly non-blocking and rearrangeable network and calculate their cost.

The input is not divisible by 16, so the number of switches that we have to use in the first layer (and consequently r) is $\lceil \frac{475}{16} \rceil = 30$. We can't have 30×30 switches in the second layer, so we will use 32×32 switches using 4 basic modules.

We differentiate the two cases:

- Strictly non-blocking: we must have $m \geq 2n - 1 = 31$. For this we will use 2 basic modules to create a 16×32 switch, so the cost will be:

- Switches:

$$\text{cost}_{\text{switch}} = 2 \cdot 30 \cdot 2 + 4 \cdot 32 = 248$$

- Links:

$$\text{cost}_{\text{links}} = 2 \cdot 32 \cdot 30 = 1920$$

- Rearrangeable: we have $m \geq n = 16$, so the cost will be:

- Switches:

$$\text{cost}_{\text{switch}} = 2 \cdot 30 + 4 \cdot 16 = 124$$

- Links:

$$\text{cost}_{\text{links}} = 2 \cdot 16 \cdot 30 = 960$$

E.4.3 Benes network

The Benes network has $2 \log n - 1$ stages of switches concatenated as a baseline followed by a reverse baseline. Can be obtained by recursively applying a 3-stage clos network, each times halving the number of inputs.

The cost of the network is:

- Switches:

$$\text{cost}_{\text{switch}} = \frac{N}{2}(2 \log N - 1)$$

- Links:

$$\text{cost}_{\text{links}} = N \cdot (2 \log N - 2)$$

Given a permutation P , we can set each switch as crossed or straight using a looping algorithm. With a permutation in the form:

$$P = \begin{pmatrix} 0 & 1 & 2 & \dots & N-1 \\ m_0 & m_1 & m_2 & \dots & m_{N-1} \end{pmatrix}$$

The looping algorithm works as follows:

1. Start from a random value i and connect the switch i to the switch m_i .
2. Continue from the switch neighbor of the switch m_i (it will be m_{i-1} or m_{i+1}), call it m_j . Connect it to the switch that in the permutation is the j such that $P(j) = m_j$.
3. Loop from the neighbor of j and start again, if at any point the loop reaches an already connected switch, start over from a free one.

Example:

Given a Benes network of size $N = 8$, compute its cost and use the looping algorithm to connect the switch following the permutation:

$$P = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 5 & 6 & 1 & 2 & 4 & 7 & 3 & 0 \end{pmatrix}$$

The cost of the network is:

- Switches:

$$\text{cost}_{\text{switch}} = \frac{8}{2}(2 \log 8 - 1) = 20$$

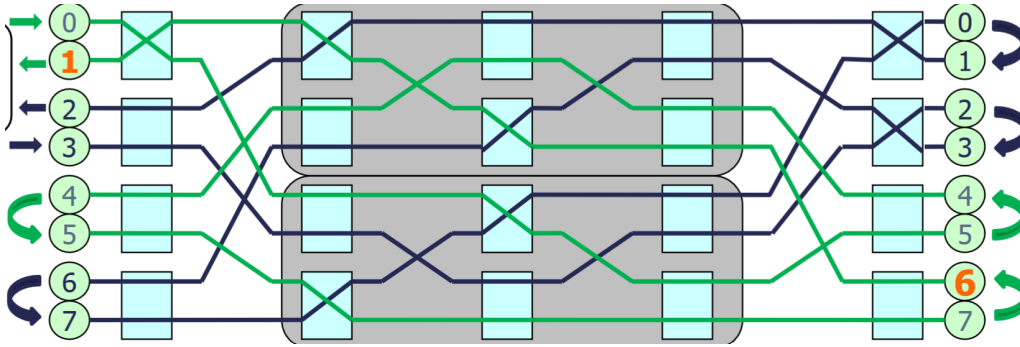
- Links:

$$\text{cost}_{\text{links}} = 8 \cdot (2 \log 8 - 2) = 32$$

The looping algorithm start from the switch 3 will work as follows:

1. Connect 3 to 2.
2. Loop going to 3 and connect it to 6.
3. Loop going to 7 and connect it to 0.
4. Loop going to 1 and connect it to 2.
5. We can't loop going to 3 because it's already connected, so we start again from 0. Connect 0 to 5.
6. Loop going to 4 and connect it to 4.
7. Loop going to 5 and connect it to 7.
8. Loop going to 6 and connect it to 1.

Now all the input and output switches are connected and the network configuration will result as:



E.4.4 Extended generalized fat tree (XGFT)

An XGFT is defined by a series of numbers in the form $XGFT(h; m_1, \dots, m_h; w_1, \dots, w_h)$, where:

- h represent the height of the tree
- m_i is the number of children nodes that a node in level i has
- w_{i+1} is the number of parents nodes that a node in level i has
- level i has m_{i+1} copies of $XGFT(i; m_1, \dots, m_i; w_1, \dots, w_i)$

The number of nodes on a generic level i are $m_{i+1} \cdot \dots \cdot m_h \cdot w_1 \cdot \dots \cdot w_i$. This can also be seen as a sliding window of size h that moves each time on the right on the array composed by $m_1, \dots, m_h, w_1, \dots, w_h$.

The XGFT is drawn by starting from the bottom or the top, calculating how many switches are on each level and connecting them based on the number of children and neighbor they have. It also exists a simple non-extended generalized fat tree, defined in the form $GFT(h; m; w)$, that is equal to the XGFT but all m and all w are equal.

Example:

Draw an $XGFT(2; 4, 4; 1, 2)$, specifying how many nodes are on each level and how many children and parents they have.

We start constructing the tree from the bottom:

1. Level 0:

- Nodes: $m_1 \cdot m_2 = 4 \cdot 4 = 16$
- Parents: $w_1 = 1$
- Children: 0

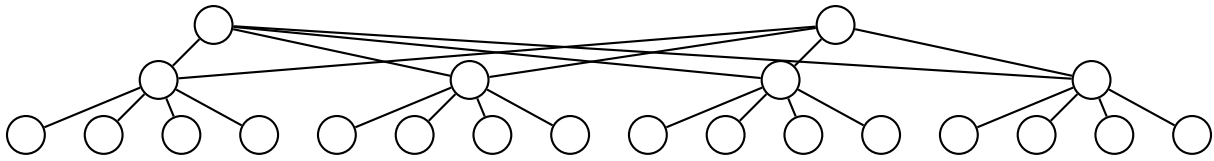
2. Level 1:

- Nodes: $m_2 \cdot w_1 = 4 \cdot 1 = 4$
- Parents: $w_2 = 2$
- Children: $m_1 = 4$

3. Level 2:

- Nodes: $w_1 \cdot w_2 = 1 \cdot 2 = 2$
- Parents: 0
- Children: $m_2 = 4$

The XGFT designed is:



E.5 Exercises on quantum computing

E.5.1 System of two qubit

Given two qubit in their superposition:

$$\begin{cases} \psi_1 = \alpha_1|0\rangle + \beta_1|1\rangle \\ \psi_2 = \alpha_2|0\rangle + \beta_2|1\rangle \end{cases}$$

The two-qubit system vector $|\psi_{12}\rangle$ is calculated by computing the vector of all states $[00,01,10,11]$, multiplying the coefficient of each qubit.

$$\psi_{12} = \begin{bmatrix} \alpha_1 \cdot \alpha_2 \\ \alpha_1 \cdot \beta_2 \\ \beta_1 \cdot \alpha_2 \\ \beta_1 \cdot \beta_2 \end{bmatrix}$$

Given the system the probability of measuring a state is equal to its magnitude squared. To control if the system (or in general any state vector) is correct, it must follow the normalization condition that means that the sum of probability of the states add up to 1.

Example:

Given the qubits:

$$\begin{cases} \psi_1 = \frac{1}{3}|0\rangle + \frac{2\sqrt{2}}{3}i|1\rangle \\ \psi_3 = \frac{\sqrt{3}}{2}|0\rangle + -\frac{1}{2}i|1\rangle \end{cases}$$

Compute the state vector of the two-qubit system, verify the normalization condition and calculate the probability of measuring $|00\rangle$ and $|10\rangle$.

The vector state of the two-qubit system is:

$$|\psi_{13}\rangle = \begin{bmatrix} \frac{1}{3} \cdot \frac{\sqrt{3}}{2} \\ \frac{1}{3} \cdot -\frac{1}{2}i \\ \frac{2\sqrt{2}}{3}i \cdot \frac{\sqrt{3}}{2} \\ \frac{2\sqrt{2}}{3}i \cdot -\frac{1}{2}i \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{3}}{6} \\ -\frac{i}{6} \\ \frac{\sqrt{6}}{3}i \\ \frac{\sqrt{2}}{3} \end{bmatrix}$$

The probability to measure each state is simply:

- $|00\rangle = \left| \frac{\sqrt{3}}{6} \right|^2 = \frac{1}{12}$
- $|01\rangle = \left| -\frac{i}{6} \right|^2 = \frac{1}{36}$
- $|10\rangle = \left| \frac{\sqrt{6}}{3}i \right|^2 = \frac{2}{3}$
- $|11\rangle = \left| \frac{\sqrt{2}}{3} \right|^2 = \frac{2}{9}$

We check the normalization condition:

$$\frac{1}{12} + \frac{1}{36} + \frac{2}{3} + \frac{2}{9} = 1$$

E.5.2 Quantum circuits

In a quantum circuit the gates are defined as matrices of size $2^n \times 2^n$ based on the number n of qubit taken as input. Given a unary gate, we can create a controlled version, where the gate is applied to the second qubit only if the first qubit is $|1\rangle$.

$$U = \begin{bmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{bmatrix} \implies CU = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & u_{00} & u_{01} \\ 0 & 0 & u_{10} & u_{11} \end{bmatrix}$$

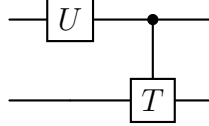
To calculate the matrix that represent a circuit we need to do a matrix multiplication between the different gates. If two gates work in parallel on two qubits the result of the system is the tensor product between the two gates. For two gates U and X , the tensor product $U \otimes X$ is:

$$U \otimes X = \begin{bmatrix} u_{00} \cdot x_{00} & u_{00} \cdot x_{01} & u_{01} \cdot x_{00} & u_{01} \cdot x_{01} \\ u_{00} \cdot x_{10} & u_{00} \cdot x_{11} & u_{01} \cdot x_{10} & u_{01} \cdot x_{11} \\ u_{10} \cdot x_{00} & u_{10} \cdot x_{01} & u_{11} \cdot x_{00} & u_{11} \cdot x_{01} \\ u_{10} \cdot x_{10} & u_{10} \cdot x_{11} & u_{11} \cdot x_{10} & u_{11} \cdot x_{11} \end{bmatrix} = \begin{bmatrix} u_{00} \cdot X & u_{01} \cdot X \\ u_{10} \cdot X & u_{11} \cdot X \end{bmatrix}$$

A matrix U is unitary if its conjugate transpose $U^\dagger$ and inverse coincide, so if $UU^\dagger = Id$. The conjugate transpose is obtained by swapping the rows and the columns and changing sign to the imaginary parts of values (i to $-i$ and vice versa).

Example:

Given the transformation U based on gates $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ and $T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{bmatrix}$:



Show what transformation U represent writing the associated 4×4 matrix and show how U acts on state represented by $[\frac{1}{2}, \frac{i}{2\sqrt{3}}, -\frac{i}{\sqrt{2}}, \frac{\sqrt{2}}{2\sqrt{3}}i]$.

The input of the gate T is the first qubit passed through X and the second one (pass through identity matrix), so the transformation U is:

$$U = CT \cdot (X \otimes Id) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\frac{\pi}{4}} \end{bmatrix} \cdot \begin{bmatrix} 0 \cdot Id & 1 \cdot Id \\ 1 \cdot Id & 0 \cdot Id \end{bmatrix}$$

The matrix representing U is:

$$U = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\frac{\pi}{4}} \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & e^{i\frac{\pi}{4}} & 0 & 0 \end{bmatrix}$$

We can check that U is unitary by calculating its conjugate transpose and checking that the product makes the identity matrix:

$$U^\dagger = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{-i\frac{\pi}{4}} \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \implies U^\dagger U = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & e^{-i\frac{\pi}{4}} \cdot e^{i\frac{\pi}{4}} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = Id$$

The result of U acting on the vector is:

$$|\psi\rangle = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & e^{i\frac{\pi}{4}} & 0 & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{2} \\ \frac{i}{2\sqrt{3}} \\ -\frac{i}{\sqrt{2}} \\ \frac{\sqrt{2}}{2\sqrt{3}}i \end{bmatrix} = \begin{bmatrix} -\frac{i}{\sqrt{2}} \\ \frac{\sqrt{2}}{2\sqrt{3}}i \\ \frac{1}{2} \\ \frac{i-1}{2\sqrt{6}} \end{bmatrix}$$